

MEMCACHED

面试 100 题 • 源码穿透版

从 KV 原理、Item / Assoc / Slab / LRU 到百万 QPS 系统设计

参考“题目 → 思路 → 深挖 → 追问 → 易错点”的面试训练结构
内容独立撰写，不复制任何第三方书籍正文

版本基准：Memcached 1.6.45 / 2026-09 官方文档与当前源码

专业面试备战手册 • 100 Questions

前言：这不是一份“八股答案合集”

Memcached 面试看似简单，真正能拉开候选人差距的地方，是能否把一个 `set foo bar` 从协议、Item、Slab、Assoc、LRU、锁一路追到底，再把单机机制外推到一致性哈希、缓存击穿和容量设计。

本书采用类似经典面试训练书的组织方式：每道题先给出面试官真正想考的能力，再给出可以在 30-90 秒内表达的核心回答，随后展开机制、源码切入点、追问和易错点。重点题会进一步拆成调用链、内存不变量或生产故障模型。

本书以 Memcached 1.6.45（2026-07-09）与 2026-09 可访问的官方文档/当前开源代码为基准。对于 segmented LRU、Meta Protocol、Extstore、Built-in Proxy 等现代特性，正文会明确区别于早期版本的传统认知。

阅读方法

先刷标记为【P0】的题，再沿 “Item → Slab → Assoc → LRU → Thread → Distributed → Cache Engineering” 主线做源码闭环。

Memcached 核心部件：Item 是中心，Assoc / Slab / LRU 分工正交

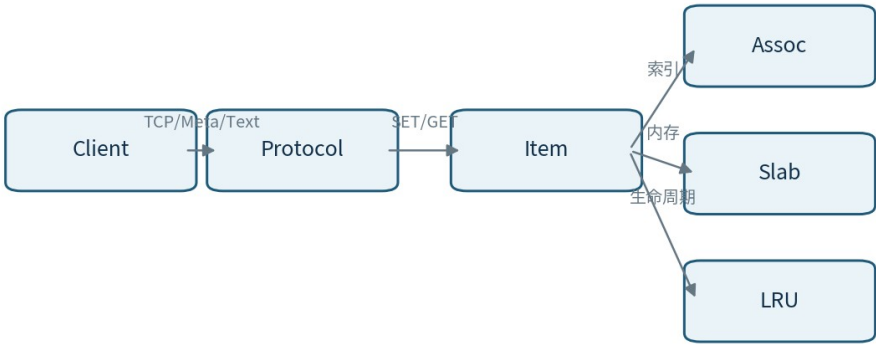


图 0-1 Memcached 核心部件关系（本书主线）

能力地图：面试官如何区分“会用”和“懂实现”

层级	能回答什么	典型题	判断标准
L1 API	GET/SET/TTL/CAS	1-10	能正确使用，不犯协议常识错误
L2 数据结构	Item/Hash/Slab/LRU	11-50	能画出指针与生命周期
L3 并发与网络	worker/libevent/locks	51-60	能解释为何高 QPS 不等于多线程

层级	能回答什么	典型题	判断标准
			程越多越好
L4 分布式	一致性哈希/节点故障	61-70	能区分路由稳定性与数据一致性
L5 生产系统	击穿/雪崩/监控	71-90	能保护 DB，而非只保护 cache
L6 架构设计	Extstore/Proxy/百万 QPS	91-100	能量化容量、故障和 SLA

目录

- 01 第一章 基础架构与设计哲学 · 第 01-010 题
- 02 第二章 Item 与 SET/GET 源码链路 · 第 11-020 题
- 03 第三章 Slab Allocator 与内存管理 · 第 21-030 题
- 04 第四章 Assoc / Hash Table · 第 31-040 题
- 05 第五章 LRU、TTL 与 Eviction · 第 41-050 题
- 06 第六章 线程、锁、Libevent 与协议 · 第 51-060 题
- 07 第七章 一致性哈希与分布式 Memcached · 第 61-070 题
- 08 第八章 缓存一致性、雪崩、击穿与热点 · 第 71-080 题
- 09 第九章 监控、容量规划与故障诊断 · 第 81-090 题
- 10 第十章 高级特性、源码与系统设计 · 第 91-100 题

30+ 道必刷核心题

- 001 Memcached 是什么？为什么它能够作为 KV 存储？
- 002 为什么说 Memcached 是 Cache，而不是 Database？
- 003 Memcached 为什么快？
- 005 Memcached 是不是一个分布式系统？
- 011 为什么 struct item 是 Memcached 最核心的数据结构？
- 012 从 set foo bar 开始，说出核心源码调用链。
- 014 新 Item 在什么时候真正对 GET 可见？
- 015 Item 为什么需要 refcount？
- 021 Memcached 为什么不用普通 malloc/free 管理所有 Item？
- 022 Page、Slab Class、Chunk、Item 的关系是什么？
- 024 Slab 解决了什么碎片，又引入了什么碎片？

- 026 为什么明明还有内存，一个 SET 仍可能发生 eviction?
- 031 assoc 模块干什么?
- 032 h_next 和 next/prev 有什么区别?
- 041 Memcached 当前还是最简单的一根 LRU 链吗?
- 043 为什么不在每一次 GET 时同步移动 LRU 节点?
- 044 TTL 到了以后 Item 会在那一秒立刻被物理删除吗?
- 046 经典“30 天 TTL 陷阱”是什么?
- 047 Expiration 和 Eviction 有什么区别?
- 051 Memcached 是“一连接一线程”吗?
- 055 Memcached 内部有哪些典型 Lock?
- 057 Memcached 现在有哪些协议?
- 061 为什么不能简单使用 $\text{hash}(\text{key}) \% N$?
- 062 一致性哈希解决什么?
- 064 一致性哈希是在 Memcached Server 内部完成的吗?
- 066 一台 Memcached Node 突然挂了会发生什么?
- 071 什么是 Cache Aside?
- 072 更新数据库与删除缓存，先做哪个?
- 075 什么叫缓存击穿 / Stampede?
- 076 什么叫缓存雪崩?
- 077 Hot Key 有什么危险?
- 081 生产 Memcached 第一眼看哪些指标?
- 083 evictions 持续增长说明什么?
- 090 为什么 Swap 对 Memcached 特别危险?
- 097 让你设计 100 万 QPS 的 Memcached 集群，怎么回答?
- 098 一个 Node 故障后，怎样避免数据库被 MISS 打死?
- 100 如果让你现场实现一个 Mini-Memcached，你怎么拆?

第一章 基础架构与设计哲学

本章目标

这一章从“系统边界和抽象”入手，建立后续源码分析的坐标系。

第 001 题 Memcached 是什么？为什么它能够作为 KV 存储？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

Hash 表把 key 映射到 item 指针，Slab 管物理内存，LRU/TTL 管生命周期；整个服务器把问题缩小为“按完整 key 快速找到 RAM 中的一段 bytes”。

深度展开

- 平均查找复杂度接近 $O(1)$ ，但碰撞时仍需沿 `h_next` 链比较真实 key。
- value 对服务端而言是原始字节，序列化语义属于客户端。
- 它的快不是单点技巧，而是从数据库能力中持续做减法。

源码 / 机制切入点

``memcached.h`` 的 ``struct item``，以及官方首页对 in-memory KV cache 的定义。

高频追问

- Hash 冲突为什么不会返回错值？
- 为什么 KV 模型比范围查询更适合 HashTable？

易错点

把 Memcached 说成“一个远程 dict”可以用于入门，但面试必须补上内存分配、淘汰和并发。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第 002 题 为什么说 Memcached 是 Cache，而不是 Database？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

它保存的是可重建数据的临时副本，而不是权威事实；数据可能因 TTL、eviction、重启或节点故障而消失。

深度展开

- 应用必须把 MISS 设计成正常路径。
- 没有 WAL/fsync/事务持久性保证。
- 把 session、计费余额等不可丢数据只存在 Memcached 是架构风险。

源码 / 机制切入点

`memcached.h` 的 `struct item`，以及官方首页对 in-memory KV cache 的定义。

高频追问

- TTL=0 是否意味着永不丢失？
- Warm Restart 是否等于持久化？

易错点

“在内存里”不是 Cache 与 Database 的核心区别，核心是可靠性语义。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第 003 题 Memcached 为什么快？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

RAM + Hash O(1) + 简单协议 + Slab + 轻量并发 + 不做持久化/事务/复杂查询，共同缩短热路径。

深度展开

- 查找通常是网络解析、hash、少量指针访问、socket write。
- Slab 将大量 malloc/free 转为 freelist O(1) 取还 chunk。
- 传统模式不承担服务端共识、复制、分片元数据。

源码 / 机制切入点

`memcached.h` 的 `struct item`，以及官方首页对 in-memory KV cache 的定义。

高频追问

- 为什么只回答“因为内存快”不够？
- 性能瓶颈可能从 CPU 转移到哪里？

易错点

高速缓存经常受网络 RTT、锁竞争、cache miss 和对象序列化限制，而非 RAM 本身。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第 004 题 Memcached 与 Redis 最根本的区别是什么？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

Memcached 聚焦通用对象缓存；Redis 是带丰富数据结构、复制、持久化及服务端计算能力的数据服务。

深度展开

- 纯 GET/SET 对象缓存、可重建数据时 Memcached 的简单性有价值。
- 需要 ZSET、Stream、持久化、服务端复制/Cluster 时 Redis 更自然。
- 不能用“Redis 单线程、Memcached 多线程”这种单一维度概括全部差异。

高频追问

- 哪些场景你会明确选 Memcached？
- Redis 也能缓存，为什么还存在 Memcached？

易错点

选择应从语义与运维复杂度出发，而不是品牌偏好。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第 005 题 Memcached 是不是一个分布式系统？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

整体部署可以是分布式缓存，但传统 Memcached server 之间默认互不感知，客户端/代理负责节点选择。

深度展开

- 每台节点维护自己的 HashTable、Slab、LRU。
- 节点不做 Raft、leader election 或自动 shard migration。
- 现代内置 proxy 改变路由位置，但不把后端变成传统强一致数据库集群。

源码 / 机制切入点

`memcached.h` 的 `struct item`，以及官方首页对 in-memory KV cache 的定义。

高频追问

- 服务器 A 知道服务器 B 吗？
- 加入新节点时谁决定 key 去哪里？

易错点

“distributed memory object caching system” 不等于 “server 端自带一致性集群协议”。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第 006 题 Memcached 的分布式逻辑到底在哪里？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

经典模式在客户端库：先把 key 映射到节点，再由节点内部 HashTable 找到 item。

深度展开

- 第一层：client-side consistent hashing。
- 第二层：server-side assoc hash。
- 因此同一个 key 的完整路径是 key → node → bucket → item。

高频追问

- 为什么可以称为“两层 Hash”？
- 路由配置不一致会怎样？

易错点

不要把 consistent hashing 与 assoc 内部 hash 混为一谈。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第 007 题 Memcached 存进去的对象到底是什么？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

对服务端来说是 key + flags + expiration + 可选 CAS + raw bytes；业务对象由客户端序列化/反序列化。

深度展开

- Item 内还有服务端私有 metadata，如 refcount、slabs_clsid、LRU 指针。
- JSON/protobuf/Java Object 只是客户端约定。
- 序列化大小会直接影响 slab class 和内存效率。

高频追问

- 为什么 flags 存在？
- 压缩阈值为什么会影响 slab 分布？

易错点

不要说 Memcached “理解 JSON”。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第 008 题 为什么 Memcached 不支持 SQL、范围查询和 JOIN？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

核心索引只优化完整 key 等值查找；加入二级索引、范围树、优化器与事务会改变系统复杂度和性能模型。

深度展开

- HashTable 对精确 key 访问最自然。
- 范围查询需要有序结构，如 B+Tree/LSM 的额外索引。
- “不支持”本身就是性能设计。

高频追问

- 如果硬要支持 prefix scan 会怎样？
- Redis 的数据结构为何能做更多操作？

易错点

面试要体现“功能越少，热路径越可预测”的设计取舍。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第 009 题 set、add、replace 的语义差异是什么？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

set 无条件存储；add 仅 key 不存在时存储；replace 仅 key 已存在时存储。

深度展开

- 三者最终都会走 item 构造与 store 决策。
- add/replace 的条件判断需要在正确锁范围内完成。
- append/prepend 又涉及重新构造 value。

高频追问

- add 能否用来做严格分布式锁？
- CAS 与 add 的并发语义有什么不同？

易错点

不要把 add 的“若不存在”误解成跨数据库事务级互斥。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第 010 题 Cache miss 后 Memcached 会自动查数据库吗？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：基础架构与设计哲学

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

不会。Memcached 返回 MISS；应用执行 DB/API 查询并决定是否回填，这是典型 Cache Aside。

深度展开

- 权威数据源在应用架构中，而不在 Memcached。
- MISS 路径必须有限流、合并请求等保护。
- 负缓存也是应用策略。

高频追问

- 为什么 miss 风暴会打垮数据库？
- 回填失败怎么处理？

易错点

把“数据库回源”说成 Memcached 内部能力是典型错误。

参考锚点：S1, S2, S4 | 本书答案按 1.6.45 / 当前官方文档校准。

第二章 Item 与 SET/GET 源码链路

本章目标

这一章沿一次真实请求进入源码，重点理解 Item 如何把多个子系统串起来。

一次 SET 的主调用链：先构造，再发布

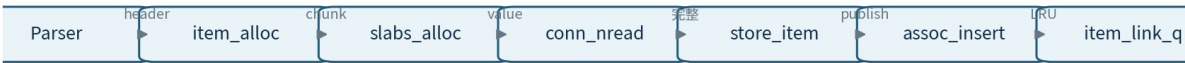


图 2-1 SET 请求：构造对象与发布对象是两个阶段

第 011 题 为什么 struct item 是 Memcached 最核心的数据结构？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

一个 item 同时承载 KV 数据、Hash 冲突链节点、LRU 双向链节点及 Slab class/lifetime metadata，是四大子系统的交汇点。

深度展开

- next/prev 受 LRU lock 保护；h_next 属于 hash chain。
- refcount 把“逻辑删除”与“物理释放”分离。
- ITEM_key/ITEM_data 通过连续内存布局减少额外对象分配。

源码 / 机制切入点

```
`proto_text.c` / `proto_parser.c` → `items.c` → `slabs.c` → `assoc.c`。
```

高频追问

- h_next 与 next/prev 为什么必须分开？
- item 为什么采用 flexible array？

易错点

若只看 key/value 字段，就错过了 Memcached 的系统设计核心。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第 012 题 从 set foo bar 开始，说出核心源码调用链。

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

文本 SET 经 parser → process_update_command → item_alloc/do_item_alloc → slabs_alloc; value 读入 ITEM_data 后，complete_nread_ascii → store_item/do_store_item → assoc_find → do_item_link/item_replace。

深度展开

- 新 item 在读取 value 前已经取得 chunk。
- 真正发布时 assoc_insert 与 item_link_q 把它同时接入 Hash 和 LRU。
- 覆盖旧值通常是 new object + replace，而不是任意原地覆盖。

源码穿透

- 文本协议的 SET 不是 “set foo bar” 一行，而是 header 与 body 两阶段：`set foo 0 0 3\r\n` 后跟 `bar\r\n`。parser 在读完 header 后已经知道 key 与 value 长度，因此能先走 item_alloc。
- `item\_alloc → do\_item\_alloc → slabs\_clsid → do\_item\_alloc\_pull/slabs\_alloc` 解决的是 “先获得最终内存”。连接随后把 `ritem` 指向 `ITEM\_data(it)`，进入 `conn\_nread`，让 body 直接落入 item。
- value 完整且 CRLF 校验通过后才进入 `store\_item`。在 key 对应 item lock 内，`do\_store\_item` 先 `do\_item\_get/assoc\_find` 判断旧对象，再选择 `do\_item\_link` 或 replace。
- `do\_item\_link` 是最接近 publication point 的位置：`assoc\_insert` 使 key 可寻址，`item\_link\_q` 使其进入 LRU，refcount 增加缓存持有引用。

源码 / 机制切入点

`proto\_text.c` / `proto\_parser.c` → `items.c` → `slabs.c` → `assoc.c`。

高频追问

- 为什么先分配再读 value？
- 哪个函数最接近 publication point？

易错点

能把这条链讲清楚，通常意味着真正看过源码而不是只背概念。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第 013 题 为什么 Memcached 先分配 Item，再读取 value？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

SET header 已知 value 长度，先分配最终 chunk 后把 socket 数据直接读向 ITEM_data，可以减少中间大 buffer 和额外 memcpy。

深度展开

- conn 结构中的 ritem 指向 item 的 value 区。
- 在完成 CRLF 校验前，新 item 尚未通过 Hash 对外可见。
- 这是 construct-before-publish 的典型并发设计。

高频追问

- 客户端半途断开会怎样？
- 超大 value 的 chunked item 又如何读取？

易错点

不要把“零拷贝”说得绝对；这里强调的是避免不必要的应用层复制。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第 014 题 新 Item 在什么时候真正对 GET 可见？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

在成功 do_item_link/replace 并插入 assoc 后才可由新 GET 找到；此前虽有物理内存，但仍是 private object。

深度展开

- 构造阶段：Slab=YES, Hash=NO, LRU=NO。
- 发布阶段：ITEM_LINKED + assoc_insert + item_link_q。
- 同一 key 的 store 决策在 item lock 范围内完成。

源码 / 机制切入点

```
`proto_text.c` / `proto_parser.c` → `items.c` → `slabs.c` → `assoc.c`。
```

高频追问

- 为什么这是类似“提交点”的概念？
- Hash 先插还是 LRU 先插有什么影响？

易错点

不要把“拿到 slab chunk”误认为“已经写入缓存”。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第 015 题 Item 为什么需要 refcount?

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

并发 GET 拿到 item 指针后，另一个线程可以把它从 Hash/LRU unlink，但必须等所有引用释放后才能复用 chunk，避免 use-after-free。

深度展开

- unlink 解决逻辑可见性；refcount 解决物理生命周期。
- SET 构造、response 输出都可能临时持有引用。
- refcount 与 item lock、LRU lock 一起维持对象安全。

并发生命周期

- 把 refcount 看成“谁还可能解引用这个 item*”。Hash/LRU unlink 只意味着未来请求不再发现它，并不意味着当前正在发送 response 的线程已经用完。
- 若删除线程在 GET 线程构造响应前立即 slabs_free，同一 chunk 可能被另一个 SET 重用，GET 就会读到完全不同对象的数据，形成典型 use-after-free。
- 因此正确顺序是：先从共享索引移除，再在最后一个临时引用释放后把 chunk 归还 slab freelist。

源码 / 机制切入点

```
`proto_text.c` / `proto_parser.c` → `items.c` → `slabs.c` → `assoc.c`。
```

高频追问

- refcount 泄漏会造成什么现象？
- 为何不能 unlink 后立即 slabs_free？

易错点

这是理解高并发缓存内存安全的必考题。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第 016 题 第二次 set foo newValue 是直接覆盖旧内存吗？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

通常先构造 new item，再在 key 锁内找到 old item，unlink old 并 link new；这使读者要么看到完整旧值，要么看到完整新值。

深度展开

- 新旧 item 在短时间内可同时物理存在。
- replace 路径维护 Hash/LRU/refcount 不变量。
- 不同 value size 可能属于不同 slab class，因此原地扩缩并不自然。

高频追问

- 如果新 value 更小为什么也不一定原地写？
- CAS 在 replace 前怎么检查？

易错点

“覆盖”是逻辑语义，不代表物理内存原地修改。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第 017 题 delete foo 在内部发生什么？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

先按 hash 找 item，再从 assoc 删除、从 LRU unlink，降低 cache 持有引用；refcount 归零后 item_free/slabs_free 回收 chunk。

深度展开

- 删除必须同时维护多个索引。

- 仍被 GET/response 持有的 item 可暂缓物理释放。
- 最终 freelist 接收归还的 chunk。

高频追问

- 只删 Hash 不删 LRU 会怎样？
- 延迟释放如何保证安全？

易错点

DELETE 是观察多索引不变量最清晰的反向路径。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第 018 题 get foo 的源码逻辑如何走？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

hash key → item lock → assoc_find → 检查过期/flush → 持有引用并构造响应；访问还可能产生 LRU bump/touch 行为。

深度展开

- GET 不是纯粹无状态读：可能更新访问信息。
- 命中后 value 通常通过 iovec/response 发送。
- 过期 item 可在访问路径上被回收。

高频追问

- 为什么 GET 也可能争锁？
- 高热点 key 对 item lock 有何影响？

易错点

“Hash 查找 $O(1)$ ” 不等于 GET 没有并发和网络成本。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第 019 题 append/prepend 为什么比普通 set 更复杂？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

value 长度变化可能需要新 item、不同 slab class 甚至 chunk chain；操作必须组合旧值与新值并安全 replace。

深度展开

- 不能假设原 chunk 后面还有连续空间。
- item size 上限仍需检查。
- 并发下必须避免基于旧值错误拼接。

高频追问

- append 是否保证列表语义？
- 大对象 append 的内存放大风险是什么？

易错点

append/prepend 是字节操作，不理解业务结构。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第 020 题 Memcached 的 CAS 是怎么工作的？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：Item 与 SET/GET 源码链路

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

gets/meta get 返回版本 token；客户端提交 cas 时只有当前 item 的 CAS 仍匹配才写入，否则返回冲突，属于乐观并发控制。

深度展开

- CAS 解决的是单个 cache item 的 read-modify-write race。
- 它不提供跨 key 事务，也不解决 DB 与 Cache 的强一致。
- Meta protocol 还支持更丰富的版本语义。

高频追问

- CAS 与数据库 MVCC 有什么相似/不同？
- ABA 问题如何理解？

易错点

不要把 CAS 等同于分布式事务。

参考锚点：S5, S6, S7, S8 | 本书答案按 1.6.45 / 当前官方文档校准。

第三章 Slab Allocator 与内存管理

本章目标

这一章回答“KV 真正存在哪里、内存满了怎么办”，是 Memcached 与普通 HashMap 的分水岭。

Slab 内存层次：Class → Page → Chunk → Item



第 021 题 Memcached 为什么不用普通 malloc/free 管理所有 Item?

面试标签

难度 P0 | 重要度 ★★★★★ | 模块: Slab Allocator 与内存管理

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

不同尺寸对象的高频分配释放会带来碎片、allocator 元数据和锁竞争；Slab 用固定尺寸 chunk 的 freelist 把常见分配降为稳定 O(1)。

深度展开

- 按大小分 class，而不是每次精确 malloc。
- 牺牲可控内部碎片换可预测性能。
- Page 可批量切成 chunk，提高局部性和管理效率。

内存模型

- Slab 的价值不是“malloc 一定慢”，而是把任意大小、任意时刻的动态分配问题变成有限个 size class 的 freelist 管理。这样 allocator 行为更稳定，也能统计每个尺寸区间的命中、淘汰和 page 分配。
- 代价是内部碎片：实际对象小于 chunk 时剩余空间无法给另一个对象使用。Growth factor 决定了“class 数量 vs 内部浪费”的基本曲线。

源码 / 机制切入点

`items.c` 的 ntotal/class 选择；`slabs.c` 的 freelist/page 分配。

高频追问

- Slab 是否完全不调用 malloc?
- 现代 allocator 已很快，为什么专用池仍有价值?

易错点

答案重点是可预测性和碎片控制，不是“malloc 很慢”这一句。

参考锚点: S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第 022 题 Page、Slab Class、Chunk、Item 的关系是什么?

面试标签

难度 P0 | 重要度 ★★★★★ | 模块: Slab Allocator 与内存管理

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

Slab Class 定义 chunk 尺寸；Page 是该 class 的一批内存；Page 被切成多个 Chunk；每个 Item 占用一个或多个 chunk。

深度展开

- 小对象常落在单 chunk。
- 大对象可采用 chunk chain。
- class 决定了对象的内存浪费和可复用范围。

源码 / 机制切入点

`items.c` 的 ntotal/class 选择；`slabs.c` 的 freelist/page 分配。

高频追问

- Page 大小是否等于 item size max？
- 为什么 class 间 chunk 不能直接混用？

易错点

把四层画出来通常比背定义更有效。

参考锚点：S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第 023 题 Slab Class 为什么需要 Growth Factor？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Slab Allocator 与内存管理

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

不可能为每个字节大小建立一个 class，因此按倍率递增 chunk size，在 class 数量和内部碎片之间折中；默认常见因子为 1.25。

深度展开

- 因子小：碎片低但 class 多。
- 因子大：class 少但浪费增加。
- 对象大小分布决定最优参数。

高频追问

- 业务对象集中在 500B 附近时怎样调优？
- 为什么改变 factor 要压测？

易错点

不能脱离 workload 讨论“最佳 growth factor”。

参考锚点：S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第 024 题 Slab 解决了什么碎片，又引入了什么碎片？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：Slab Allocator 与内存管理

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

它显著减少不可预测的外部碎片，但会产生 $\text{chunk_size} - \text{item_size}$ 的内部碎片。

深度展开

- 101B item 落入 120B chunk，会浪费约 19B。
- 大量对象时几个字节差异也会放大为 GB。
- chunked item 让大对象 class 设计更灵活。

源码 / 机制切入点

`items.c` 的 `ntotal/class` 选择；`slabs.c` 的 `freelist/page` 分配。

高频追问

- 如何通过 `stats sizes` 评估？
- 压缩 value 为什么可能反而改善 slab 效率？

易错点

“Slab 消灭碎片” 是错误表述。

参考锚点：S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第 025 题 如何根据 Item 大小选择 Slab Class？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Slab Allocator 与内存管理

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

先算完整 `ntotal`，包括 item header、CAS、key、flags metadata、value、CRLF 等，再找能容纳它的最小 class。

深度展开

- value size 不等于 item size。
- 编译选项和 flags 会影响 header。
- 边界大小跨一个 class 会突然增加内存占用。

高频追问

- 250B value 最终一定进 256B chunk 吗？
- 如何设计对象序列化避免跨 class 边界？

易错点

面试中若直接用 value 长度选 class，说明没有理解 item 布局。

参考锚点：S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第 026 题 为什么明明还有内存，一个 SET 仍可能发生 eviction？

面试标签

难度 P1 | 重要度 ★★★★★ | 模块：Slab Allocator 与内存管理

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

空闲内存可能集中在别的 slab class；目标 class 没有 free chunk 且不能获得新 page 时，只能从其 LRU 回收或等待 slab reassignment。

深度展开

- 这是“全局有空间、局部 class 紧张”的经典现象。
- 对象大小分布变化会造成 slab stagnation。
- 现代 automove 会尝试重新平衡 page。

源码 / 机制切入点

`items.c` 的 ntotal/class 选择；`slabs.c` 的 freelist/page 分配。

高频追问

- 为什么全局 bytes 指标不够？
- 如何用 per-slab eviction 定位？

易错点

这是生产诊断中非常常见的反直觉问题。

参考锚点：S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第 027 题 Slab Reassign / Automove 是什么？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Slab Allocator 与内存管理

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

后台把 page 从富余 class 迁到紧张 class，缓解工作负载对象尺寸变化后的内存错配。

深度展开

- 迁移 page 时需处理其中仍有效/被引用的 item。
- automove 根据 eviction、age、free ratio 等信号做决策。
- 它减少过去必须重启才能重新分布内存的场景。

高频追问

- 迁移为什么不能瞬间完成？
- busy item 如何处理？

易错点

Slab reassignment 是内存资源再平衡，不是 key shard migration。

参考锚点：S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第 028 题 Memcached 如何处理很大的 Item？

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块：Slab Allocator 与内存管理

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

现代实现可使用 item_chunk 链把较大对象跨多个 chunk 组装；item header 持有/关联 chunk chain。

深度展开

- 这避免为了大 item 把 slab class 设计得过度稀疏。
- 大 value 仍会增加网络、锁持有、内存放大和 tail latency。
- 调整 item_size_max 不等于“大对象就适合缓存”。

高频追问

- chunk chain 对 GET 发送有什么影响？

- 为什么大对象会伤害命中率？

易错点

知道“可以更大”不等于建议缓存超大对象。

参考锚点：S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第 029 题 -m 4G 是否表示进程 RSS 绝对不会超过 4GB？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Slab Allocator 与内存管理

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

不是。-m 主要约束 item storage；进程还有 HashTable、connection/read buffer、线程、libevent、TLS、proxy/extstore metadata 等额外内存。

深度展开

- 连接数很大时网络 buffer 也可显著占内存。
- 操作系统和 allocator 还会造成 RSS 与逻辑使用量差异。
- 容量规划应留出 headroom。

高频追问

- 为什么容器 memory limit 需要比 -m 大？
- RSS 超限时 OOM Killer 会怎样？

易错点

把 -m 当作进程硬上限容易导致线上 OOM。

参考锚点：S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第 030 题 Memcached 最大 Item 是不是固定 1MB？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：Slab Allocator 与内存管理

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

1MB 是常见默认限制，不是不可改变的协议常量；当前源码允许通过 item_size_max/-l 调整，并支持更大的上限。

深度展开

- 提升上限会改变内存效率和网络尾延迟。
- 大对象使用 chunked item。
- 默认值与“合理生产值”是两个问题。

高频追问

- 为什么官方默认仍偏保守？
- 最大对象调到 64MB 会带来哪些风险？

易错点

“永远只能 1MB”是过时的八股答案。

参考锚点：S5, S8, S10 | 本书答案按 1.6.45 / 当前官方文档校准。

第四章 Assoc / Hash Table

本章目标

这一章拆解单节点的逻辑索引：HashTable 如何找到 Item，以及多索引如何维持不变量。

第 031 题 assoc 模块干什么？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：Assoc / Hash Table

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

Assoc 维护 key → item* 的逻辑寻址，是服务器本地 HashTable；它不拥有业务 value 的独立副本。

深度展开

- assoc_find/insert/delete 是主要操作。
- HashTable 与 Slab 的“位置”概念不同：一个是逻辑索引，一个是物理内存。
- 同一 item 通过指针被多个结构引用。

逻辑位置 vs 物理位置

- Assoc 回答“给定 foo，哪个 item* 是它”；Slab 回答“这个 item* 占 RAM 的哪块 chunk”。两套位置相互正交。HashTable resize 可以改变逻辑 bucket，但无需移动 item payload 的物理地址。
- 这也是为什么 HashTable 只存指针：多个索引围绕一个对象协作，而不是复制数据。

源码 / 机制切入点

`assoc.c` 的 find/insert/delete 与 `item.h\_next`。

高频追问

- 为什么不直接把 value 放 bucket？
- HashTable resize 时 item 会搬内存吗？

易错点

Assoc 是理解 KV 查找路径的核心。

参考锚点：S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第 032 题 h_next 和 next/prev 有什么区别？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：Assoc / Hash Table

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

h_next 连接 Hash collision chain；next/prev 连接 LRU 双向链。二者受到不同锁和生命周期规则保护。

深度展开

- 一个 item 可以同时在这两套链表中。
- Hash 删除不等于 LRU 删除。
- 字段分离是维持多索引不变量的基础。

源码 / 机制切入点

`assoc.c` 的 find/insert/delete 与 `item.h\_next`。

高频追问

- 为什么不能复用 next 做 hash chain？
- 并发锁顺序为什么重要？

易错点

这是源码识别题，通常要求秒答。

参考锚点：S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第 033 题 Memcached 怎么处理 Hash Collision？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：Assoc / Hash Table

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

多个 key 落到同一 bucket 后沿 h_next 遍历，并比较 key 长度与真实 key bytes，因此 hash 相同不会被视为同一个 key。

深度展开

- 理想情况下链很短。
- 冲突过多会导致指针追踪与 cache miss。
- HashTable 扩容用于控制平均链长。

高频追问

- 最坏复杂度是多少？
- 恶意 key 是否可能构造 Hash DoS？

易错点

O(1) 是平均复杂度而不是数学保证。

参考锚点：S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第 034 题 HashTable 为什么不能无限小？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Assoc / Hash Table

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

bucket 数太少会提升 load factor 和 collision chain 长度，使 GET 从少量内存访问变成多次 pointer chasing。

深度展开

- CPU cache locality 会恶化。
- 锁持有时间也可能增长。
- 扩容本身又要处理并发迁移成本。

高频追问

- 如何选择初始 hashpower？
- 对象量骤增时会发生什么？

易错点

性能问题不仅看 big-O，还要看 cache line 和锁竞争。

参考锚点：S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第 035 题 Rehash 为什么可能成为性能问题？

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块：Assoc / Hash Table

面试官在考什么

是否能将源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

若一次性停顿并搬迁全部 bucket，会在大缓存上造成明显延迟；Memcached 采用后台/渐进扩容思想减小请求热路径阻塞。

深度展开

- 扩容期间可能同时存在 old/new table 状态。

- 迁移必须保持 find/insert/delete 正确。
- 历史版本专门优化过 hash expansion contention。

高频追问

- 渐进 rehash 如何保证新请求能找到旧 bucket?
- 扩容线程和 item lock 如何协作?

易错点 讲到“后台扩容”比只说“HashMap 会 resize”更有深度。

参考锚点: S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第 036 题 Internal Hash 与 Consistent Hash 是同一个东西吗?

面试标签 难度 P0 | 重要度 ★★★★★☆ | 模块: Assoc / Hash Table

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话 不是: consistent hashing 解决 key → server; assoc hash 解决 server 内 key → bucket。

深度展开

- 前者是集群路由策略。
- 后者是单节点数据结构。
- 可使用不同 hash 算法或实现。

高频追问

- 为什么客户端 Hash 变化会造成 MISS，但 assoc resize 不会?
- 两层 Hash 各自优化什么?

易错点 把两者混淆是 Memcached 分布式面试的常见失分点。

参考锚点: S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第 037 题 如果 Hash 函数分布不好会怎样?

面试标签 难度 P1 | 重要度 ★★★☆☆ | 模块: Assoc / Hash Table

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

热点 bucket、长冲突链、更多 CPU cache miss 和更长 item lock 持有时间，最终表现为吞吐下降与 P99 升高。

深度展开

- 分布质量与计算速度都重要。
- Hash DoS 是安全与性能交叉问题。
- 不要只看均值，要看 bucket 尾部分布。

高频追问

- Murmur/Jenkins 的取舍是什么？
- Hash 算法切换会影响客户端路由吗？

易错点

服务端内部 Hash 与客户端路由 Hash 要分别讨论。

参考锚点：S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第 038 题 为什么 HashTable 存 item* 而不是复制 Value?

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：Assoc / Hash Table

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

数据只保存一份，Hash 与 LRU 都通过指针引用同一 item，减少内存与一致性维护成本。

深度展开

- 复制 value 会使更新/删除要同步多份。
- item 的连续布局让元数据与 payload 局部性更好。
- refcount 确保被引用期间内存不被复用。

高频追问

- 指针方案的风险是什么？
- 为什么需要严格生命周期管理？

易错点

节省内存的代价是必须非常谨慎地处理 dangling pointer。

参考锚点：S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第 039 题 为什么删除 Item 时必须同时处理 Assoc 与 LRU?

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块: Assoc / Hash Table

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

一个是可寻址性，一个是淘汰队列成员关系；只修改其中一边会留下 zombie node 或让已“删除”的对象仍可 GET。

深度展开

- 多索引结构需要共同维护 invariant。
- refcount 负责第三层物理生命周期。
- do_item_unlink 是观察这些关系的关键入口。

高频追问

- 如何设计锁顺序避免死锁？
- 异常路径如何保证不会漏 unlink？

易错点

系统代码的难点往往是“不变量”，不是单个算法。

参考锚点: S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第 040 题 现场实现一个 Mini-Memcached HashTable 怎么做?

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块: Assoc / Hash Table

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

先做 bucket array + separate chaining，支持 find/insert/delete/resize，再增加锁分片、refcount 与渐进 rehash。

深度展开

- Item 同时保留 h_next 与 lru_prev/lru_next。
- HashTable 只保存指针。
- 测试需覆盖 collision、delete head/middle/tail、resize 并发。

高频追问

- 为什么选择 chaining 而非开放寻址？
- 如何做分片锁？

易错点

面试 coding 重点是清晰不变量，不需要复刻全部生产优化。

参考锚点：S5, S7 | 本书答案按 1.6.45 / 当前官方文档校准。

第五章 LRU、TTL 与 Eviction

本章目标

这一章关注对象何时值得继续留在 RAM，区分 TTL、reclaim 与 eviction，并理解现代 segmented LRU。

第 041 题 Memcached 当前还是最简单的一根 LRU 链吗？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：LRU、TTL 与 Eviction

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

不是。现代 Memcached 默认采用 segmented LRU 思路，按 HOT/WARM/COLD 等区域组织，并由后台 maintainer 调整对象，减少高 QPS 下严格 LRU 的锁争用。

深度展开

- 传统“Hash + 一根双向链”适合解释基础，但不是当前完整实现。
- 1.5 时代引入 segmented LRU、crawler、slab automove。
- 近似策略用一点精确度换吞吐和命中率。

现代 LRU

- 教科书 strict LRU 在每次 GET 都移动链表头，会把热点读转化为共享写；在多核高 QPS 下，LRU lock 和 cache line bouncing 会成为瓶颈。
- Segmented LRU 的关键不是 HOT/WARM/COLD 三个名字，而是把“新对象”“被证明会复用的对象”“候选淘汰对象”分离，并把部分重排交给后台线程。

源码 / 机制切入点

`items.c` 的 link/unlink、LRU maintainer/crawler; 1.5.0 release notes。

高频追问

- 为什么 segmented LRU 更适合真实 workload？
- TEMP LRU 是什么？

易错点

如果只背“访问就移到 LRU 头部”，容易被源码追问击穿。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第 042 题 HOT/WARM/COLD 分别解决什么问题？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：LRU、TTL 与 Eviction

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

HOT 收纳新/近期对象，WARM 保留被证明有复用价值的对象，COLD 作为主要淘汰候选；后台根据访问标记与年龄在区间间迁移。

深度展开

- 防止一次性扫描流量污染整个缓存。
- 真正有价值的对象可从 COLD 得到提升。
- 各段占比可配置并受 tail age 约束。

高频追问

- 新对象为什么不直接进入永久 HOT？
- 怎样识别 scan pollution？

易错点

理解目标比死记百分比更重要。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第 043 题 为什么不在每一次 GET 时同步移动 LRU 节点？

面试标签

难度 P2 | 重要度 ★★★★★ | 模块：LRU、TTL 与 Eviction

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

严格 LRU 每 GET 都要修改共享双向链并加锁，在百万 QPS 下会成为严重写热点；Memcached 采用访问标记/限频 bump/后台维护降低争用。

深度展开

- GET 逻辑上是读，但 LRU touch 让它变成共享状态写。
- ITEM_UPDATE_INTERVAL 等机制抑制频繁重排。
- 近似 LRU 常比严格 LRU 的实际吞吐更好。

源码 / 机制切入点

`items.c` 的 link/unlink、LRU maintainer/crawler；1.5.0 release notes。

高频追问

- 这和 CPU cache coherence 有什么关系？
- 为什么 TinyLFU 等算法也常采用近似统计？

易错点

高性能系统要警惕“为了完美策略付出过高维护成本”。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第 044 题 TTL 到了以后 Item 会在那一秒立刻被物理删除吗？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：LRU、TTL 与 Eviction

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

不一定。过期首先意味着逻辑上不可命中；物理空间可在 GET 发现、分配扫描或 crawler 后台遍历时回收。

深度展开

- 这避免维护高成本全局定时器/每秒扫描。
- 因此 curr_items/bytes 与 “有效未过期对象” 可能短暂不一致。
- 后台 crawler 改善长期占用。

源码 / 机制切入点

`items.c` 的 link/unlink、LRU maintainer/crawler; 1.5.0 release notes。

高频追问

- 为什么 lazy expiration 是好设计？
- 大量相同 TTL 会对 crawler 有什么压力？

易错点

过期与释放必须区分。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第 045 题 LRU Crawler 是干什么的？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：LRU、TTL 与 Eviction

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

后台沿 LRU 检查过期对象并 reclaim，减少无人访问的 expired item 长期占 chunk，也

可提供 metadump 等能力。

深度展开

- 它把部分回收工作从请求路径移到后台。
- 扫描速率需要限流，避免争锁和 CPU 峰值。
- crawler stats 可用于观察回收效率。

高频追问

- crawler 太激进会怎样？
- 它和 evict tail 的职责区别？

易错点

Crawler 是“主动清过期”，不是新的淘汰算法。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第 046 题 经典“30 天 TTL 陷阱”是什么？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：LRU、TTL 与 Eviction

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

Basic Text 协议中较短 exptime 按相对秒解释，超过 30 天则按 Unix timestamp 绝对时间解释，因此直接传 31 天秒数可能得到完全不同的日期。

深度展开

- 客户端库通常会封装，但底层语义必须知道。
- 0 表示不因 TTL 过期。
- Meta protocol 可提供更丰富的 TTL 元信息。

源码 / 机制切入点

`items.c` 的 link/unlink、LRU maintainer/crawler; 1.5.0 release notes。

高频追问

- 为什么设计成这种历史兼容规则？
- 跨时区有影响吗？

易错点

这是生产中真实出现过的坑。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第 047 题 Expiration 和 Eviction 有什么区别？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：LRU、TTL 与 Eviction

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

Expiration 是业务 TTL 到期；Eviction 是内存压力下提前牺牲仍有效 item。TTL=0 只是不自动过期，仍可被 eviction。

深度展开

- evicted 指标说明工作集/容量问题。
- reclaimed 更偏向回收已过期对象。
- 二者对应用的 MISS 原因不同。

源码 / 机制切入点

`items.c` 的 link/unlink、LRU maintainer/crawler；1.5.0 release notes。

高频追问

- 怎么用 stats 区分？
- 永不过期对象过多会怎样？

易错点

不要把所有 MISS 都归因于 TTL。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第 048 题 刚被 GET 的 Item 会不会同时被另一个线程 Evict？

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块：LRU、TTL 与 Eviction

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

可以先从 Hash/LRU 逻辑 unlink，但只要 GET/response 仍持有 refcount，物理 chunk 不能被重新分配。

深度展开

- future GET 看不到它。

- 当前 GET 仍能安全完成 response。
- 最后引用释放时才回 Slab。

高频追问

- 这类似哪些内存回收技术？
- 为什么 refcount 仍需锁配合？

易错点 逻辑删除与物理回收分离是核心。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第 049 题 什么叫 noeviction？有什么风险？

面试标签 难度 P1 | 重要度 ★★★☆☆ | 模块：LRU、TTL 与 Eviction

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话 关闭 eviction 后内存不足时新写入直接失败，能保护已有 cache 但降低可写性；适合极少数需要明确失败语义的场景。

深度展开

- 应用必须处理 NOT_STORED/NO_MEMORY。
- 不淘汰并不使数据持久。
- 容量变化时可能造成持续写失败。

高频追问

- 为什么数据库 buffer pool 与 cache 对 eviction 的偏好不同？
- 哪些 workload 会考虑 noeviction？

易错点 默认缓存通常更愿意牺牲旧 item 保持可写。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第 050 题 为什么要看每个 slab class 的 eviction，而不只看全局？

面试标签 难度 P1 | 重要度 ★★★☆☆ | 模块：LRU、TTL 与 Eviction

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

全局 evictions 可能掩盖局部 class 热点；将 per-slab evicted 与 total_pages、get_hits、cmd_set 对齐才能判断是真 OOM 还是 slab 分布失衡。

深度展开

- 同一实例不同对象尺寸的压力完全不同。
- Automove 的存在也说明 per-class 资源是关键。
- 容量规划应结合对象 size histogram。

高频追问

- 哪个指标最能证明“加内存有效”？
- 怎样识别某个序列化版本导致对象变大？

易错点

生产诊断必须从全局下钻到 class。

参考锚点：S5, S6, S10, S11 | 本书答案按 1.6.45 / 当前官方文档校准。

第六章 线程、锁、Libevent 与协议

本章目标

这一章进入高 QPS 的工程核心：event loop、worker、锁粒度与现代协议。

第 051 题 Memcached 是“一连接一线程”吗？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：线程、锁、Libevent 与协议

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

不是。典型架构是监听线程接入连接，多个 worker 各自运行 libevent event loop，同时管理大量 socket。

深度展开

- 线程数远小于连接数。
- worker 还有配套 utility/background 线程。
- 这样避免每连接线程的栈内存和上下文切换。

I/O 与并发

- Memcached 的 worker 不是阻塞等一个连接完成，而是每个 worker 拥有 event base，维护很多 conn 状态机。连接可能在 parsing、nread、write 等状态之间切换。
- 这使线程数量与连接数量解耦：几万连接并不意味着几万线程，系统资源主要转向 fd、buffer 与 event registration。

源码 / 机制切入点

`thread.c`、`conn` 状态机与 Protocol 文档。

高频追问

- 连接如何分配到 worker？
- 为什么 epoll/kqueue 适合这种模型？

易错点

不要把多线程等同于 blocking thread-per-connection。

参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第 052 题 Libevent 在 Memcached 中解决什么问题？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：线程、锁、Libevent 与协议

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

它抽象 epoll/kqueue 等 readiness notification，让 worker 只处理 ready socket，并把连接状态机挂到事件循环。

深度展开

- 高并发连接无需高并发线程。
- 状态机负责 read/parse/nread/write 等阶段。
- 网络层效率决定 KV 热路径上限。

高频追问

- 为什么 event-driven 仍可能被大 value 阻塞？
- 边缘触发/水平触发有什么区别？

易错点

Libevent 解决 I/O multiplexing，不自动解决所有 CPU/锁问题。

参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第 053 题 新连接是怎么交给 Worker 的？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：线程、锁、Libevent 与协议

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

监听线程 accept 后选择 worker，通过队列/notifier 把 fd 交给对应 event loop，worker 创建 conn 并注册事件。

深度展开

- 跨线程通知常用 pipe/eventfd。
- 连接后续通常固定由该 worker 管理。
- 负载分配策略影响连接均衡。

高频追问

- 为什么不让所有 worker 同时 accept？
- SO_REUSEPORT 能否改变架构？

易错点

回答要体现 event loop 与 ownership。

参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第 054 题 为什么 Memcached 不是线程越多越快？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：线程、锁、Libevent 与协议

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

线程增加到一定程度后会遭遇 lock contention、context switch、cache coherence、NUMA 和 scheduler 开销，吞吐可能反而下降。

深度展开

- worker 数应与核心、网络、workload 匹配。
- 共享 Slab/LRU/Hash 元数据造成不可消除的协调成本。
- P99 往往比平均 QPS 更早恶化。

高频追问

- 如何压测选择 -t？
- 超线程核心该怎么算？

易错点

“核心数多少就开多少线程”只是起点。

参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第 055 题 Memcached 内部有哪些典型 Lock？

面试标签

难度 P1 | 重要度 ★★★★★★ | 模块：线程、锁、Libevent 与协议

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

主要包括 item locks、LRU locks、slabs lock 及连接/统计等同步结构；不同字段明确由不同锁保护。

深度展开

- item lock 常按 hash 分片。
- LRU lock 保护 next/prev 等队列关系。
- slabs lock 保护 freelist/page 管理。

源码 / 机制切入点

`thread.c`、`conn` 状态机与 Protocol 文档。

高频追问

- 锁顺序不一致会怎样？
- 为什么 per-thread stats 能减少争用？

易错点

能说出“字段由哪把锁保护”是源码级答案。

参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第 056 题 为什么不能只用一个 Global Mutex？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：线程、锁、Libevent 与协议

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

所有 GET/SET/DELETE 串行化会让多核退化为单核，并在热点下产生 convoy；细粒度/分片锁允许不相关 key 并行。

深度展开

- 粒度越细，锁管理复杂度越高。
- 还需避免死锁和内存回收 race。
- 后台线程降低请求路径上的共享写。

高频追问

- 为什么无锁 HashTable 不一定更快？
- 锁分片数量怎么选？

易错点

系统设计永远有协调复杂度与吞吐的折中。

参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第 057 题 Memcached 现在有哪些协议？

面试标签

难度 P1 | 重要度 ★★★★★★ | 模块：线程、锁、Libevent 与协议

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

官方当前主要支持 Basic Text 与 Meta Text；Binary Protocol 已 deprecated，不再作为新客户端优先方案。

深度展开

- Meta 与 Text 可交叉兼容。
- Meta 用 flags/tokens 提供更高扩展性。
- 协议选择影响功能、RTT 与客户端复杂度。

源码 / 机制切入点

`thread.c`、`conn` 状态机与 Protocol 文档。

高频追问

- 为什么 Binary 曾经流行？
- 旧客户端如何迁移 Meta？

易错点

这是版本敏感题，不能沿用十年前答案。

参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第 058 题 为什么 Binary Protocol 被 Deprecated？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：线程、锁、Libevent 与协议

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

官方总结包括安全问题、效率不理想、扩展性有限、SASL 支持麻烦及客户端实现复杂；Meta Protocol 提供更简单可扩展替代。

深度展开

- Binary 的固定 packet header 并未自动带来更高实际效率。
- Meta 只需少量命令 + 可选 flags。
- 新功能更多向 Meta 演进。

高频追问

- 文本协议为什么反而可以更高效？
- 协议可扩展性如何设计？

易错点

“二进制一定比文本快”是典型误区。

参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第 059 题 Meta Protocol 解决了什么传统 GET/SET 难表达的问题？

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块：线程、锁、Libevent 与协议

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

它把 CAS、TTL metadata、stale、anti-dogpiling、stale-while-revalidate、opaque token、binary key 等能力编码为可组合 flags，减少多轮请求。

深度展开

- mg/ms/md/ma 等小命令集覆盖主要操作。
- 客户端只请求需要返回的 metadata，降低 wire bytes。
- 特别适合高级缓存一致性模式。

高频追问

- 如何用 Meta 做 lease？
- 为什么 stale-while-revalidate 能减 stampede？

易错点

Meta 是现代 Memcached 高阶面试的区分度题。

参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第 060 题 为什么 Multi-get 通常比循环 get() 好？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：线程、锁、Libevent 与协议

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

批量请求能减少网络 RTT、syscall、packet 和客户端调度成本；但在大集群中需要按目标节点拆分并并发请求。

深度展开

- 单节点 text get 原生支持多个 key。

- 跨节点 multiget 由客户端按 shard 聚合。
- 大批量也可能导致 head-of-line 和巨型 response。

高频追问

- batch size 多大合理？
- 什么是 multiget hole？

易错点

网络往返往往比本地 Hash lookup 贵得多。

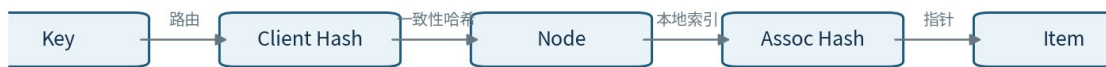
参考锚点：S4, S5, S9 | 本书答案按 1.6.45 / 当前官方文档校准。

第七章 一致性哈希与分布式 Memcached

本章目标

这一章把单机 KV 外推为集群：路由、扩缩容、节点故障和现代 proxy。

分布式 Memcached 的两层 Hash：客户端选节点，节点内选 bucket



第 061 题 为什么不能简单使用 $\text{hash}(\text{key}) \% N$?

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：一致性哈希与分布式 Memcached

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

节点数 N 改变时模数变化会重映射绝大多数 key，造成缓存整体失温和数据库回源风暴。

深度展开

- $\%N$ 只适合节点集合稳定或可接受全量 remap 的场景。
- 缓存扩缩容是常态，因此需要更稳定路由。
- 同一客户端配置必须一致。

为什么 $\%N$ 会雪崩

- 如果 N 改为 $N+1$ ，`hash % N` 的余数空间整体变化，大多数 key 会被映射到新节点；这些 key 在新节点都没有缓存，于是“扩容”反而瞬间制造海量 MISS。
- 一致性哈希的真正价值是控制 membership change 的扰动范围；它解决的是路由稳定性而非数据强一致。

源码 / 机制切入点

官方 Architecture/Proxy 文档；客户端 consistent hashing 语义。

高频追问

- N 从 100 到 101 大概会影响多少 key?
- 如果使用 rendezvous hashing 呢?

易错点

核心问题是 membership change 的迁移比例。

参考锚点：S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第 062 题 一致性哈希解决什么?

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：一致性哈希与分布式 Memcached

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

把节点和 key 映射到环/哈希空间，节点加入或退出时只让邻近范围 key 改变归属，降低重映射比例。

深度展开

- 目标不是消除所有迁移，而是最小化。
- 仍需解决负载均衡和节点权重。
- 缓存数据通常 lazy warm，不做主动数据搬迁。

源码 / 机制切入点

官方 Architecture/Proxy 文档；客户端 consistent hashing 语义。

高频追问

- 为什么一致性哈希不是强一致算法？
- 环上节点分布不均怎么办？

易错点

名字里的“一致性”指路由稳定性，不是数据一致性。

参考锚点：S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第 063 题 Virtual Node 是干什么的？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：一致性哈希与分布式 Memcached

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

一个物理节点映射多个虚拟点，让哈希空间更均匀，也便于按机器容量设置权重。

深度展开

- 虚拟点越多，分布方差通常越小。
- 但路由表和重构成本会增加。
- 现代客户端也可能采用 Ketama/Rendezvous 等不同策略。

高频追问

- heterogeneous node 如何加权？
- 虚拟节点数量越多越好吗？

易错点

回答要谈均衡与开销的折中。

参考锚点：S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第 064 题 一致性哈希是在 Memcached Server 内部完成的吗？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：一致性哈希与分布式 Memcached

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

传统模式不是，客户端库维护节点列表并计算路由；服务器收到请求后只处理本地 key。

深度展开

- 这让 server 端保持简单。
- 也把 membership 一致性责任推给客户端配置。
- 内置 proxy 可把路由集中到代理层。

源码 / 机制切入点

官方 Architecture/Proxy 文档；客户端 consistent hashing 语义。

高频追问

- 客户端升级不同步会发生什么？
- proxy 有什么运维优势？

易错点

这是经典陷阱题。

参考锚点：S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第 065 题 加一台 Memcached 后，旧数据会自动迁过去吗？

面试标签

难度 P0 | 重要度 ★★★★☆ | 模块：一致性哈希与分布式 Memcached

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

传统模式不会主动迁移；路由变化后部分 key 去新节点产生 MISS，再由应用回源并 lazy warm。

深度展开

- 这减少了数据搬迁机制复杂度。
- 代价是扩容瞬间 origin 压力上升。
- 可用 warmup、双写或渐进流量缓解。

高频追问

- 如何做无冲击扩容？
- 为什么缓存比数据库更适合 lazy migration？

易错点

缓存可重建是这一设计成立的前提。

参考锚点：S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第 066 题 一台 Memcached Node 突然挂了会发生什么？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：一致性哈希与分布式 Memcached

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

该节点上的缓存副本立即不可用，客户端会 MISS/重路由；真正风险是大量回源把 DB/API 推过容量阈值。

深度展开

- 缓存节点故障属于“放大器”故障。
- 需要连接超时、熔断、request coalescing、DB 限流。
- 是否重新哈希到其他 cache 节点取决于客户端策略。

源码 / 机制切入点

官方 Architecture/Proxy 文档；客户端 consistent hashing 语义。

高频追问

- 为什么 cache SLA 不能只看 cache 自己？
- 如何估算一个节点故障的 DB 增量 QPS？

易错点

面试官常看你是否想到下游保护。

参考锚点：S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第 067 题 Memcached 原生是不是自动复制两份？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：一致性哈希与分布式 Memcached

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

传统核心 server 没有默认复制机制；应用、客户端或现代 proxy 可自行设计复制/多池，但那不是单节点核心语义。

深度展开

- 复制会降低有效缓存容量。
- 对可重建数据，接受 MISS 有时比全量复制更经济。
- 关键热点或昂贵对象可选择性复制。

高频追问

- 为什么缓存复制策略应与数据库不同？
- 多副本如何处理写一致性？

易错点

不要把 Redis replication 的知识套到 Memcached。

参考锚点：S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第 068 题 为什么缓存系统不一定需要强复制？

面试标签

难度 P1 | 重要度 ★★★★★ | 模块：一致性哈希与分布式 Memcached

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

因为对象可由 authoritative source 重建，强复制的 RAM 与写放大成本可能高于偶发 MISS 的代价；真正要保护的是回源容量与恢复过程。

深度展开

- 是否复制取决于 rebuild cost 和 origin headroom。
- 热门 key、昂贵计算结果可能值得额外副本。
- 多 AZ 延迟也会影响方案。

高频追问

- 怎样做 selective replication?
- 什么时候缓存也需要强可用?

易错点

要用经济模型而非口号回答。

参考锚点: S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第 069 题 两个客户端的 Hash Ring 配置不一致会怎样?

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块: 一致性哈希与分布式 Memcached

面试官在考什么

是否理解实现机制与工程取舍, 而不仅是 API 或结论。

30-90 秒核心回答

一句话

同一 key 可能被路由到不同节点, 造成重复缓存、互相 MISS、命中率下降甚至读到不同版本副本。

深度展开

- 节点列表顺序、权重、hash 算法、虚拟点参数都需一致。
- 部署滚动升级时要特别谨慎。
- 集中 proxy 可降低客户端配置漂移。

高频追问

- 如何做 ring versioning?
- 灰度扩容如何避免双环长期共存?

易错点

分布式缓存的控制面往往比数据面更容易被忽视。

参考锚点: S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第 070 题 Built-in Proxy 是否改变了传统模型?

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块: 一致性哈希与分布式 Memcached

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来, 解释“为什么这样设计”。

30-90 秒核心回答

一句话

是。自 1.6.23 起可选内置 proxy 让客户端连接前端代理, 由代理做 pool/routing/fallback/复制模式, 后端仍是普通 Memcached。

深度展开

- 可将 smart client 复杂度集中到服务端。
- proxy 支持 Basic/Meta 并以配置脚本描述路由。
- 代理也成为新的容量与故障域，需要 HA。

高频追问

- proxy 与独立 mcrouter 类工具有何差异？
- proxy 是否让后端自动强一致？

易错点

集中路由不等于分布式数据库共识。

参考锚点：S1, S14 | 本书答案按 1.6.45 / 当前官方文档校准。

第八章 缓存一致性、雪崩、击穿与热点

本章目标

这一章站到应用侧看缓存：一致性、穿透、击穿、雪崩与 Hot Key。

第 071 题 什么是 Cache Aside?

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：缓存一致性、雪崩、击穿与热点

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

应用先读 Cache，MISS 时读权威数据源并回填；写通常先更新 DB，再 invalidate cache，让下一次读重建。

深度展开

- 缓存不是数据库透明层，而是应用主动控制。
- MISS path 需要保护。
- 删除而非更新缓存通常简化并发一致性。

源码 / 机制切入点

Basic/Meta Protocol；应用 Cache Aside 与 stale/anti-dogpiling 语义。

高频追问

- 为什么不是 Memcached 自动回源？
- 写穿 Write-through 有何不同？

易错点

这是最基础也是最容易继续深挖的缓存模式。

参考锚点：S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 072 题 更新数据库与删除缓存，先做哪个？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：缓存一致性、雪崩、击穿与热点

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

经典 Cache Aside 通常先提交 DB，再删除 cache；若反过来，删除后并发 reader 可能回源读旧 DB 并把旧值重新缓存。

深度展开

- 任何跨两系统的两步操作仍存在失败窗口。

- 可用 CDC/outbox、版本号、短 TTL、重试等加强。
- 强一致要求要明确业务容忍度。

源码 / 机制切入点

Basic/Meta Protocol; 应用 Cache Aside 与 stale/anti-dogpiling 语义。

高频追问

- DB 成功但 delete cache 失败怎么办？
- 延迟双删是否真的严格正确？

易错点 不要把某个顺序说成数学上绝对无 race。

参考锚点：S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 073 题 为什么通常推荐 DELETE Cache，而不是 UPDATE Cache？

面试标签 难度 P1 | 重要度 ★★★☆☆ | 模块：缓存一致性、雪崩、击穿与热点

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话 invalidate 将多个并发写入问题简化为“缓存失效后重新从唯一权威源加载”，避免在 DB 与 Cache 中同时计算和写同一业务状态。

深度展开

- 更新 cache 需要序列化规则与 DB 完全一致。
- 并发 writer 顺序可能造成旧 cache 覆盖新 cache。
- 删除后仍需考虑回填 race。

高频追问

- 哪些场景会主动 update cache？
- 如何做版本检查防旧值回填？

易错点 原则是减少双写状态，而不是教条。

参考锚点：S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 074 题 什么叫缓存穿透？

面试标签 难度 P0 | 重要度 ★★★★★ | 模块：缓存一致性、雪崩、击穿与热点

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

大量请求访问本来就不存在的 key，Cache MISS 后持续打到 DB；常用负缓存、Bloom filter、输入校验、限流。

深度展开

- 负缓存 TTL 不宜过长，否则新建数据短期不可见。
- Bloom filter 有假阳性但无假阴性（正确构造前提下）。
- 攻击流量还需安全层限流。

高频追问

- 如何防止随机 key 攻击？
- 负缓存如何处理权限差异？

易错点

不能只回答 Bloom filter。

参考锚点：S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 075 题 什么叫缓存击穿 / Stampede?

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：缓存一致性、雪崩、击穿与热点

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

一个高热 key 失效后，大量并发请求同时 MISS 并回源；解决核心是让“一个请求重建，其他请求等待或读 stale”。

深度展开

- singleflight/request coalescing。
- lease/锁 + double check。
- stale-while-revalidate、TTL jitter、提前刷新。

Stampede 的正确抽象

- 问题不是“某个 key 过期”本身，而是很多并发请求同时发现“需要昂贵重建”。因此治理目标是把 N 个重建工作压成 1 个，或让 N-1 个请求暂时使用 stale 数据。
- 最常见组合是 singleflight + stale-while-revalidate + TTL jitter；若只加互斥锁但没有超时、失败恢复与 stale fallback，锁服务故障仍会扩大事故。

源码 / 机制切入点

Basic/Meta Protocol; 应用 Cache Aside 与 stale/anti-dogpiling 语义。

高频追问

- 锁服务本身挂了怎么办？
- Meta protocol 如何做 anti-dogpiling？

易错点

这是高并发缓存面试必考。

参考锚点：S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 076 题 什么叫缓存雪崩？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：缓存一致性、雪崩、击穿与热点

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

大量 key 同时失效或整个 cache 集群不可用，使回源流量在短时间内成倍增加。

深度展开

- 随机化 TTL 只解决同步过期，不解决整集群故障。
- 需要多级缓存、降级、DB 限流、熔断、warmup。
- 容量规划要按“cache fail-open”场景压测。

源码 / 机制切入点

Basic/Meta Protocol; 应用 Cache Aside 与 stale/anti-dogpiling 语义。

高频追问

- 如何做故障演练？
- 缓存全挂时哪些接口先降级？

易错点

雪崩治理是系统工程而非单个 TTL 技巧。

参考锚点：S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 077 题 Hot Key 有什么危险？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：缓存一致性、雪崩、击穿与热点

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

一致性哈希只做 key 级分片，一个超级热 key 仍集中到一台节点，造成单机 CPU/网络/item lock 热点。

深度展开

- L1 本地缓存可吸收热点。
- 可做 key replica/多副本读，但写与一致性更复杂。
- 热点检测应看 per-key 或 proxy sampling。

源码 / 机制切入点

Basic/Meta Protocol；应用 Cache Aside 与 stale/anti-dogpiling 语义。

高频追问

- 如何无侵入发现 hot key？
- 复制热点 key 时如何选择副本？

易错点

“加节点”不能自动解决单 key 热点。

参考锚点：S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 078 题 CAS 能不能解决缓存与数据库的强一致性？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：缓存一致性、雪崩、击穿与热点

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

不能。CAS 只对 Memcached item 的版本竞争提供乐观控制，它不知道 DB transaction；跨系统仍需 CDC、version、事务消息等架构。

深度展开

- CAS 能防止旧客户端覆盖新 cache。
- 但 DB 已更新、cache 未更新仍是另一类问题。
- external CAS/version 可辅助而非替代跨系统一致性。

高频追问

- 如何用 DB row version 映射到 cache?
- 为什么双写问题本质上难?

易错点

不要把单对象原子性放大为跨系统事务。

参考锚点: S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 079 题 incr/decr 为什么适合做轻量计数器?

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块: 缓存一致性、雪崩、击穿与热点

面试官在考什么

是否理解实现机制与工程取舍, 而不仅是 API 或结论。

30-90 秒核心回答

一句话

服务端原子地解析无符号整数并增减, 避免客户端 GET-modify-SET 的 lost update; 但数据仍会因 eviction/restart 丢失。

深度展开

- decr 不低于 0。
- 不存在的 key 默认失败。
- 适合统计/限时计数, 不适合财务真账。

高频追问

- 计数器如何初始化避免 race?
- 跨节点全局计数怎么做?

易错点

原子命令不等于持久化强一致计数器。

参考锚点: S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 080 题 Session 能不能存在 Memcached?

面试标签

难度 P0 | 重要度 ★★★★★ | 模块: 缓存一致性、雪崩、击穿与热点

面试官在考什么

能否给出准确、结构化、不过时的核心答案, 并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

可以, 但前提是 session 丢失可被业务接受或有其他恢复机制; 节点故障、eviction、restart 都可能让用户被迫重新登录。

深度展开

- 安全/权限信息应谨慎处理。
- 高价值 session 可采用持久 backing store。
- TTL 与会话过期语义要一致。

高频追问

- 购物车是否适合只存在 Memcached?
- 如何做 graceful session loss?

易错点

先讨论数据丢失后果，再讨论性能。

参考锚点：S2, S3, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第九章 监控、容量规划与故障诊断

本章目标

这一章聚焦线上可观测性：如何从 miss、eviction、slab 和连接指标还原真实故障。

第 081 题 生产 Memcached 第一眼看哪些指标？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：监控、容量规划与故障诊断

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

先看 hit/miss、cmd_get/cmd_set、bytes/limit、curr_items、evictions/reclaimed、connections/listen_disabled，再按 slab 下钻，并结合 CPU、RSS、网络与 P99。

深度展开

- 缓存指标必须和 origin DB/API 指标联看。
- 率比累计值更有诊断价值。
- 版本发布前后要做时间序列对比。

源码 / 机制切入点

``stats`` / ``stats items`` / ``stats slabs`` 与 Maintenance 文档。

高频追问

- 哪个指标最能提前预警雪崩？
- 为什么只看 hit rate 不够？

易错点

监控应该回答“容量、命中、延迟、故障域”四类问题。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第 082 题 Hit Rate 怎么计算？

面试标签

难度 P0 | 重要度 ★★★★★☆ | 模块：监控、容量规划与故障诊断

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

通常 $\text{get_hits} / (\text{get_hits} + \text{get_misses})$ ；但全局高 hit rate 可能掩盖少数极昂贵 MISS 或单个 slab/class 问题。

深度展开

- 还应看 miss cost。
- 按业务 key namespace 统计更有意义。
- 写多读少的缓存即便 hit rate 低也未必错误。

高频追问

- 99% hit rate 是否一定优秀？
- 如何计算缓存节省的 DB CPU？

易错点

指标必须和业务代价结合。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第 083 题 evictions 持续增长说明什么？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：监控、容量规划与故障诊断

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

说明有仍有效 item 因内存压力被提前淘汰，但原因可能是整体容量不足、特定 class 紧张、对象 size 变化或 slab 分布失衡。

深度展开

- 先看 per-slab evicted。
- 再看 total_pages、age、hit、set。
- 不要立即加机器而不确认 workload。

源码 / 机制切入点

`stats` / `stats items` / `stats slabs` 与 Maintenance 文档。

高频追问

- evictions=0 是否表示内存足够？
- noeviction 模式下怎么判断压力？

易错点

诊断要从结果追到机制。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第 084 题 为什么 Item 过期后 curr_items 可能没有立即下降？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：监控、容量规划与故障诊断

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

TTL 到期主要改变逻辑有效性，lazy expiration 意味着物理对象可能等到访问、crawler 或分配回收时才被 unlink/free。

深度展开

- 因此统计有时间滞后。
- crawler 可改善。
- 不要用 curr_items 直接代表“有效业务 key 数”。

高频追问

- bytes 也会滞后吗？
- 如何统计真实 TTL 分布？

易错点

理解 lazy expiration 才能正确读监控。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第 085 题 get_misses 突然暴涨，你怎么排查？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：监控、容量规划与故障诊断

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

先排节点故障/路由变更，再看 key format/TTL/deploy/flush，随后看 evictions/slab imbalance、流量结构、warmup 和客户端序列化 bug。

深度展开

- 把时间点与发布/扩缩容事件对齐。
- 对比所有节点而非平均值。
- 检查 DB 同时是否出现 QPS/latency 抬升。

高频追问

- 如何快速区分“路由错”与“内存不足”？
- 某一节点 miss 特别高说明什么？

易错点

排查顺序比背一堆可能性更重要。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第 086 题 listen_disabled_num 很高说明什么？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：监控、容量规划与故障诊断

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

表示曾因连接容量限制停止/拒绝接收新连接；要检查 maxconns、fd limit、连接泄漏、客户端不复用连接及连接 churn。

深度展开

- 短连接会显著放大 accept/handshake 成本。
- 系统 ulimit 与 Memcached 配置需匹配。
- 连接高峰可能先于 CPU 打满。

高频追问

- 为什么连接池过大也可能伤害？
- TIME_WAIT 与 server 连接有什么关系？

易错点

连接是独立容量维度。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第 087 题 为什么应该使用 Persistent Connections？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：监控、容量规划与故障诊断

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

KV 本地操作极快，反复 TCP handshake/teardown 的成本会占据总延迟；连接池和长连接减少 RTT、syscall 与内核状态开销。

深度展开

- 还需设置合理 idle timeout。
- 连接池应防止每进程无限扩张。
- 跨 AZ RTT 比服务器处理时间大得多。

高频追问

- 什么时候短连接反而合理？
- TLS 会进一步放大什么成本？

易错点

缓存性能优化经常从客户端网络开始。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第 088 题 stats slabs 能帮助判断什么？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：监控、容量规划与故障诊断

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

观察每 class 的 chunk size、pages、free/used chunks、set/get/eviction 等，定位对象大小分布和局部内存压力。

深度展开

- 与 stats items 结合看 LRU age/evicted。
- 发布导致对象膨胀时会看到 class 流量迁移。
- 可用于评估 growth factor 与压缩策略。

高频追问

- 如何识别“对象刚好跨 class 边界”？
- 为什么 total_pages 特别重要？

易错点

这是内存调优最有用的诊断入口之一。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第 089 题 Memcached 在 NUMA 机器上有什么问题？

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块：监控、容量规划与故障诊断

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

worker 访问远端 NUMA 内存会增加 latency 和 cache coherence 开销；极端低延迟场景可按 NUMA node 绑定进程、CPU 与内存。

深度展开

- 单大实例并不总优于每 NUMA 一实例。
- NIC affinity、IRQ 也会影响尾延迟。
- 必须用实际 workload benchmark。

高频追问

- numactl 如何验证改善？
- 多实例会怎样影响 consistent hash？

易错点

NUMA 是高阶性能调优题，不应在小规模系统过度优化。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第 090 题 为什么 Swap 对 Memcached 特别危险？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：监控、容量规划与故障诊断

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

Memcached 的价值来自可预测 RAM latency；一旦 item page 被 swap，GET 会触发 page fault 和磁盘访问，P99 可上升几个数量级。

深度展开

- 更大的“逻辑缓存”不等于更好。
- 应控制系统内存 headroom 和 swappiness。
- 容器 OOM 与 swap 策略也需理解。

源码 / 机制切入点

`stats` / `stats items` / `stats slabs` 与 Maintenance 文档。

高频追问

- 为什么宁可更小的无 swap cache?
- Extstore 与 swap 有什么本质区别?

易错点

Extstore 是显式分层设计，swap 是不可控内核换页。

参考锚点：S11, S12 | 本书答案按 1.6.45 / 当前官方文档校准。

第十章 高级特性、源码与系统设计

本章目标

最后一章把所有机制组合为高级特性与系统设计题，目标是形成架构级表达。

第 091 题 Extstore 是什么？Memcached 不是纯 RAM 吗？

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块：高级特性、源码与系统设计

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

Extstore 用 RAM 保存索引/metadata/热数据，将部分较大或冷 value 放 SSD/NVMe，扩展有效容量；它仍然是缓存，不是突然变成持久数据库。

深度展开

- 磁盘访问由专门 I/O 线程处理。
- RAM 仍决定可索引对象数。
- SSD endurance、队列深度与 compaction 会影响性能。

高频追问

- Extstore 与 OS swap 的差异是什么？
- 什么对象适合下沉 SSD？

易错点

显式 cache tiering 与透明 paging 是完全不同的控制模型。

参考锚点：S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 092 题 Extstore 为什么仍需要大量 RAM？

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块：高级特性、源码与系统设计

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

HashTable、item header、LRU、location metadata、network buffer 等仍在内存；SSD 主要承载 value bulk，因此 RAM 是控制面与索引容量。

深度展开

- 小对象 metadata 比例可能很高。
- 容量规划需要同时算 header RAM 和 SSD value。
- 热点 value 可 recache 回 RAM。

高频追问

- 如何估算 10 亿 key 的 metadata RAM?
- value 很小时 Extstore 是否值得?

易错点

“SSD 容量大”并不会消除 RAM 索引瓶颈。

参考锚点: S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 093 题 什么是 Warm Restart?

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块: 高级特性、源码与系统设计

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来, 解释“为什么这样设计”。

30-90 秒核心回答

一句话

特定配置下进程重启可以尽可能复用已有缓存内存状态, 降低重启后的 cold-cache 风暴, 但它不提供数据库式持久化承诺。

深度展开

- 目标是运维可用性而非数据可靠性。
- 机器断电/内存丢失仍会失去 cache。
- 版本/配置兼容性需遵循官方要求。

高频追问

- Warm Restart 和 replication 谁解决的问题不同?
- 为什么不能把它当持久存储?

易错点

要区分“减少冷启动”与“保证数据不丢”。

参考锚点: S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 094 题 Meta Protocol 怎么实现 stale-while-revalidate?

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块: 高级特性、源码与系统设计

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来, 解释“为什么这样设计”。

30-90 秒核心回答

一句话

允许过期/标 stale 对象在限定窗口继续被部分请求使用, 同时通过 token/flags 让一个客户端承担刷新, 其他客户端避免一起回源。

深度展开

- 核心是 request coalescing + stale serving。
- 可显著降低热 key stampede。
- 需要客户端正确实现状态机。

高频追问

- 刷新客户端失败怎么办？
- stale window 太长有什么风险？

易错点

Meta 的价值在复杂缓存语义，而不是换一种命令拼写。

参考锚点：S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 095 题 Meta CAS Override 有什么用途？

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块：高级特性、源码与系统设计

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

客户端可把外部版本号映射为 cache CAS，例如 DB row version/CRC，用于拒绝旧版本写入、比较多池状态和做修复。

深度展开

- 它提升版本协同能力，但仍不是跨系统事务。
- 版本必须单调/可比较才有意义。
- 错误版本策略会造成活锁或长期旧值。

高频追问

- 时间戳能否直接当版本？
- 多数据中心 clock skew 怎么办？

易错点

外部版本是工具，不是自动一致性。

参考锚点：S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 096 题 Built-in Proxy 为什么值得关注？

面试标签

难度 P2 | 重要度 ★★★☆☆ | 模块：高级特性、源码与系统设计

面试官在考什么

是否能把源码细节、性能模型和系统设计联系起来，解释“为什么这样设计”。

30-90 秒核心回答

一句话

它把路由、pool、fallback 和部分复制策略从大量客户端库集中到 Memcached 前端，简化多语言客户端和动态路由。

深度展开

- 前端仍需要水平扩展和高可用。
- 代理增加一跳但可换取控制面集中。
- Meta 能力可穿过 proxy 使用。

高频追问

- proxy 会不会成为瓶颈？
- 如何做无状态化配置发布？

易错点

任何集中层都要讨论容量和故障域。

参考锚点：S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 097 题 让你设计 100 万 QPS 的 Memcached 集群，怎么回答？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：高级特性、源码与系统设计

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

先定 QPS/读写比、平均和 P99 item size、working set、命中目标、hot-key skew、TTL 分布、origin miss capacity；节点数取内存、CPU、网络、QPS 四个约束的最大值。

深度展开

- 采用 client/proxy consistent routing，必要时 L1 local cache。
- 按单节点压测得出的安全 QPS，而不是理论带宽，计算 N+1 故障容量。
- 设计热点、stampede、rolling deploy、扩缩容、监控和回源限流。

系统设计答题框架

- 第一步量化工作集： $\text{RAM_required} \approx \text{unique_hot_items} \times (\text{avg_item_total_size} + \text{allocator_overhead}) / \text{target_utilization}$ 。注意 avg_item_total_size 包含 key/header/CAS/内部碎片，而不是只算 value。
- 第二步量化节点：分别按 RAM、network Gbps、CPU QPS、P99 SLA 计算节点数，取最大值；再按 N+1 或节点故障比例留容量。
- 第三步验证失效场景：若一个节点承载 5% 命中流量，故障后这 5% 的请求会向 origin 暴露多少额外 QPS？若 DB 只有 10% headroom，就必须先做 stale/L1/coalescing/限流，而不是只谈加 cache 节点。

源码 / 机制切入点

Meta、Extstore、Proxy 文档；结合前九章做系统设计。

高频追问

- 如何估算内存 headroom？
- 一台节点宕机会多出多少 DB QPS？

易错点

系统设计先问约束，再画架构；不要先拍脑袋说“20 台”。

参考锚点：S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 098 题 一个 Node 故障后，怎样避免数据库被 MISS 打死？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：高级特性、源码与系统设计

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

组合 request coalescing、origin 并发限流、熔断、stale serving、L1 cache、warm spare/replica、渐进 rehash 和业务降级，让恢复流量受控。

深度展开

- 失败时优先保护权威数据源。
- 所有客户端同时 failover 可能形成 thundering herd。
- 恢复节点也要渐进 warm，避免 cache 填充抢占 DB。

源码 / 机制切入点

Meta、Extstore、Proxy 文档；结合前九章做系统设计。

高频追问

- 如果 DB headroom 只有 10% 怎么办？
- 哪些数据可以直接返回 stale？

易错点

缓存灾备的核心是“控制回源”，不是让 cache 100% 不故障。

参考锚点：S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 099 题 什么场景下会明确选择 Memcached，而不是 Redis？

面试标签

难度 P1 | 重要度 ★★★☆☆ | 模块：高级特性、源码与系统设计

面试官在考什么

是否理解实现机制与工程取舍，而不仅是 API 或结论。

30-90 秒核心回答

一句话

需求主要是可重建 raw-object GET/SET、TTL、CAS、计数，且不需要持久化、复杂数据结构、服务端事务/复制时，Memcached 的简单性和可预测内存模型很有优势。

深度展开

- 决策要考虑团队生态和客户端成熟度。
- 如果需求正在向 queue/ZSET/stream/lock 扩张，Redis 更自然。
- 不要为了“性能传说”忽略运维和功能需求。

高频追问

- Memcached 的 Slab 相对 Redis allocator 有什么特点？
- 为什么云厂商有时更偏 Redis？

易错点

优秀答案是条件式选择，不是站队。

参考锚点：S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

第 100 题 如果让你现场实现一个 Mini-Memcached，你怎么拆？

面试标签

难度 P0 | 重要度 ★★★★★ | 模块：高级特性、源码与系统设计

面试官在考什么

能否给出准确、结构化、不过时的核心答案，并能承接至少 1-2 层追问。

30-90 秒核心回答

一句话

按 7 个阶段：HashMap GET/SET/DELETE → Item+TTL → 自研 Hash collision → LRU → Slab pool → refcount/locks → TCP event loop/protocol；每阶段用测试维护不变量。

深度展开

- 先保证语义，再做内存池和并发。
- Item 同时有 h_next 与 lru_prev/lru_next。
- SET 使用 construct-before-publish，DELETE 使用 unlink-before-free。

实现路线

- 第 1-3 阶段只建立正确语义：GET/SET/DELETE、TTL、Hash collision；第 4-5 阶段引入 LRU 和 Slab，第一次出现“一个对象属于多结构”的系统不变量。
- 第 6 阶段加入 refcount 和锁，重点测试 GET 与 DELETE/SET 并发；第 7 阶段再加 TCP/event loop，把网络状态机与 KV core 分离。
- 最关键的两个工程约束：SET 必须 construct-before-publish；DELETE 必须 unlink-before-free。

源码 / 机制切入点

Meta、Extstore、Proxy 文档；结合前九章做系统设计。

高频追问

- 如何测试并发 use-after-free？
- 如何模拟 slab imbalance？

易错点

这是把 100 题知识真正闭环成工程能力的终极题。

参考锚点：S3, S13, S14, S15 | 本书答案按 1.6.45 / 当前官方文档校准。

附录 A：100 题速查表

题号	题目	难度	必刷
001	Memcached 是什么？为什么它能够作为 KV 存储？	P0	★
002	为什么说 Memcached 是 Cache，而不是 Database？	P0	★
003	Memcached 为什么快？	P0	★
004	Memcached 与 Redis 最根本的区别是什么？	P0	
005	Memcached 是不是一个分布式系统？	P0	★
006	Memcached 的分布式逻辑到底在哪里？	P0	
007	Memcached 存进去的对象到底是什么？	P0	
008	为什么 Memcached 不支持 SQL、范围查询和 JOIN？	P0	
009	set、add、replace 的语义差异是什么？	P0	
010	Cache miss 后 Memcached 会自动查数据库吗？	P0	
011	为什么 struct item 是 Memcached 最核心的数据结构？	P0	★
012	从 set foo bar 开始，说出核心源码调用链。	P0	★
013	为什么 Memcached 先分配 Item，再读取 value？	P1	
014	新 Item 在什么时候真正对 GET 可见？	P0	★
015	Item 为什么需要 refcount？	P0	★
016	第二次 set foo newValue 是直接覆盖旧内存吗？	P1	
017	delete foo 在内部发生什么？	P1	

题号	题目	难度	必刷
018	get foo 的源码逻辑如何走?	P1	
019	append/prepend 为什么比普通 set 更复杂?	P1	
020	Memcached 的 CAS 是怎么工作的?	P0	
021	Memcached 为什么不用普通 malloc/free 管理所有 Item?	P0	★
022	Page、Slab Class、Chunk、Item 的关系是什么?	P0	★
023	Slab Class 为什么需要 Growth Factor?	P1	
024	Slab 解决了什么碎片, 又引入了什么碎片?	P0	★
025	如何根据 Item 大小选择 Slab Class?	P1	
026	为什么明明还有内存, 一个 SET 仍可能发生 eviction?	P1	★
027	Slab Reassign / Automove 是什么?	P1	
028	Memcached 如何处理很大的 Item?	P2	
029	-m 4G 是否表示进程 RSS 绝对不会超过 4GB?	P1	
030	Memcached 最大 Item 是不是固定 1MB?	P0	
031	assoc 模块干什么?	P0	★
032	h_next 和 next/prev 有什么区别?	P0	★
033	Memcached 怎么处理 Hash Collision?	P0	
034	HashTable 为什么不能无限小?	P1	
035	Rehash 为什么可能成为性能问题?	P2	
036	Internal Hash 与 Consistent Hash 是同一个东西吗?	P0	

题号	题目	难度	必刷
037	如果 Hash 函数分布不好会怎样?	P1	
038	为什么 HashTable 存 item* 而不是复制 Value?	P1	
039	为什么删除 Item 时必须同时处理 Assoc 与 LRU?	P1	
040	现场实现一个 Mini-Memcached HashTable 怎么做?	P1	
041	Memcached 当前还是最简单的一根 LRU 链吗?	P0	★
042	HOT/WARM/COLD 分别解决什么问题?	P1	
043	为什么不在每一次 GET 时同步移动 LRU 节点?	P2	★
044	TTL 到了以后 Item 会在那一秒立刻被物理删除吗?	P0	★
045	LRU Crawler 是干什么的?	P1	
046	经典“30 天 TTL 陷阱”是什么?	P0	★
047	Expiration 和 Eviction 有什么区别?	P0	★
048	刚被 GET 的 Item 会不会同时被另一个线程 Evict?	P2	
049	什么叫 noeviction? 有什么风险?	P1	
050	为什么要看每个 slab class 的 eviction, 而不只看全局?	P1	
051	Memcached 是“一连接一线程”吗?	P0	★
052	Libevent 在 Memcached 中解决什么问题?	P0	
053	新连接是怎么交给 Worker 的?	P1	
054	为什么 Memcached 不是线程越多越快?	P0	
055	Memcached 内部有哪些典型 Lock?	P1	★

题号	题目	难度	必刷
056	为什么不能只用一个 Global Mutex?	P0	
057	Memcached 现在有哪些协议?	P1	★
058	为什么 Binary Protocol 被 Deprecated?	P1	
059	Meta Protocol 解决了什么传统 GET/SET 难表达的问题?	P2	
060	为什么 Multi-get 通常比循环 get() 好?	P0	
061	为什么不能简单使用 $\text{hash}(\text{key}) \% N$?	P0	★
062	一致性哈希解决什么?	P0	★
063	Virtual Node 是干什么的?	P0	
064	一致性哈希是在 Memcached Server 内部完成的吗?	P0	★
065	加一台 Memcached 后, 旧数据会自动迁过去吗?	P0	
066	一台 Memcached Node 突然挂了会发生什么?	P0	★
067	Memcached 原生是不是自动复制两份?	P0	
068	为什么缓存系统不一定需要强复制?	P1	
069	两个客户端的 Hash Ring 配置不一致会怎样?	P1	
070	Built-in Proxy 是否改变了传统模型?	P2	
071	什么是 Cache Aside?	P0	★
072	更新数据库与删除缓存, 先做哪个?	P0	★
073	为什么通常推荐 DELETE Cache, 而不是 UPDATE Cache?	P1	
074	什么叫缓存穿透?	P0	
075	什么叫缓存击穿 / Stampede?	P0	★

题号	题目	难度	必刷
076	什么叫缓存雪崩?	P0	★
077	Hot Key 有什么危险?	P0	★
078	CAS 能不能解决缓存与数据库的强一致性?	P1	
079	incr/decr 为什么适合做轻量计数器?	P1	
080	Session 能不能存在 Memcached?	P0	
081	生产 Memcached 第一眼看哪些指标?	P0	★
082	Hit Rate 怎么计算?	P0	
083	evictions 持续增长说明什么?	P0	★
084	为什么 Item 过期后 curr_items 可能没有立即下降?	P1	
085	get_misses 突然暴涨, 你怎么排查?	P1	
086	listen_disabled_num 很高说明什么?	P1	
087	为什么应该使用 Persistent Connections?	P0	
088	stats slabs 能帮助判断什么?	P1	
089	Memcached 在 NUMA 机器上有什么问题?	P2	
090	为什么 Swap 对 Memcached 特别危险?	P0	★
091	Extstore 是什么? Memcached 不是纯 RAM 吗?	P2	
092	Extstore 为什么仍需要大量 RAM?	P2	
093	什么是 Warm Restart?	P2	
094	Meta Protocol 如何实现 stale-while-revalidate?	P2	
095	Meta CAS Override 有什么用途?	P2	

题号	题目	难度	必刷
096	Built-in Proxy 为什么值得关注?	P2	
097	让你设计 100 万 QPS 的 Memcached 集群, 怎么回答?	P0	★
098	一个 Node 故障后, 怎样避免数据库被 MISS 打死?	P0	★
099	什么场景下会明确选择 Memcached, 而不是 Redis?	P1	
100	如果让你现场实现一个 Mini-Memcached, 你怎么拆?	P0	★

附录 B：7 天冲刺计划

日期	范围	目标
Day 1	1-20	基础 + Item + SET/GET; 必须能白板画 `set foo bar` 主调用链。
Day 2	21-40	Slab + Assoc; 手写 Class/Page/Chunk 与 h_next/LRU 指针图。
Day 3	41-60	LRU/TTL + Worker/Lock/Protocol; 重点理解严格 LRU 为什么在高 QPS 下不划算。
Day 4	61-80	一致性哈希 + 缓存工程; 口述节点故障、击穿、雪崩、Hot Key。
Day 5	81-100	监控 + 高级特性 + 系统设计; 完成百万 QPS 设计题。
Day 6	核心题复盘	只刷带 ★ 的必刷题; 每题控制在 90 秒内回答, 再接受两轮追问。
Day 7	模拟面试	随机抽 20 题 + 2 道系统设计; 所有答案必须包含“机制 + 取舍 + 生产影响”。

附录 C：面试表达模板

模板 1

先给结论: Memcached 的 X 本质是……; 然后拆三层: 数据结构、并发不变量、生产

模板 2

影响。

如果面试官追源码：入口函数 → 核心对象 → 关键锁 → publication/unlink 点 → 失败/回收路径。

模板 3

如果面试官追系统设计：先量化 workload → 找内存/CPU/网络/SLA 约束 → 设计 N+1 → 再处理热点、MISS 与 origin 保护。

附录 D：官方资料与源码索引

S1 Memcached 官方首页与 Downloads (1.6.45, 2026-07-09)

<https://www.memcached.org/>

S2 Basic Text Protocol

<https://docs.memcached.org/protocols/basic/>

S3 Meta Text Protocol

<https://docs.memcached.org/protocols/meta/>

S4 Protocols Overview (Binary deprecated)

<https://docs.memcached.org/protocols/>

S5 memcached.h (struct item / conn / flags)

<https://github.com/memcached/memcached/blob/master/memcached.h>

S6 items.c (item alloc/link/unlink/LRU)

<https://github.com/memcached/memcached/blob/master/items.c>

S7 assoc.c (HashTable)

<https://github.com/memcached/memcached/blob/master/assoc.c>

S8 slabs.c / slabs.h (Slab allocator)

<https://github.com/memcached/memcached/blob/master/slabs.c>

S9 thread.c (worker / locks)

<https://github.com/memcached/memcached/blob/master/thread.c>

S10 Memcached 1.5.0 Release Notes (segmented LRU / crawler / automove)

<https://docs.memcached.org/releasenotes/releasenotes150/>

S11 Server Maintenance (hit rate / evictions / slabs)

<https://docs.memcached.org/serverguide/maintenance/>

S12 Server Configuring (threads / connections / memory)

<https://docs.memcached.org/serverguide/configuring/>

S13 Flash Storage / Extstore

<https://docs.memcached.org/features/flashstorage/>

S14 Built-in Proxy

<https://docs.memcached.org/features/proxy/>

S15 FAQ

<https://docs.memcached.org/userguide/faq/>

版本说明：Memcached 官方 Downloads 在本书制作时列出的 latest stable 为 1.6.45（2026-07-09）。源码链接使用官方 GitHub 当前 master 作为函数定位辅助；稳定版与 master 的少量细节可能随提交变化，面试应优先掌握不变量与设计意图。

真正吃透 Memcached，只需要抓住四个问题

Assoc	怎么找到它？
Slab	它住在哪里？
LRU / TTL	它还能活多久？
refcount / locks	并发下何时可以安全释放？

然后把这四个问题从单机外推到集群、故障和系统设计。